

Functional Requirements

Authentication

- User **Sign Up**
- User **Login**
- JWT-based authentication
- Protected APIs (unauthenticated access should fail)

Task Management (CRUD)

Each task should have:

- Title (required)
- Description
- Status (Pending, In Progress, Completed)
- Created At

Features:

- Create a task
- View task list
- Edit a task
- Delete a task
- Only logged-in users can manage their own tasks

Frontend Requirements

- Login & Signup pages
- Dashboard to list tasks
- Form to add/edit tasks
- Proper loading & error handling
- Clean and readable UI

- Responsive layout (basic)

Backend Requirements

- REST APIs:
 - POST /auth/signup
 - POST /auth/login
 - GET /tasks
 - POST /tasks
 - PUT /tasks/:id
 - DELETE /tasks/:id
- Input validation
- Proper HTTP status codes
- JWT middleware for protected routes

Bonus Points (Optional)

- Pagination or search on tasks
- Role-based access (Admin/User)
- Password hashing (bcrypt)
- API documentation (Swagger / Postman collection)

Task: Build a Real-Time Notification System

Objective

Design and implement a scalable real-time notification system with React (frontend) and Node.js (backend).

◇ Backend Requirements (Node.js)

1. Create REST APIs:
 - a. Create notification
 - b. Get user notifications (with pagination)
 - c. Mark as read
 - d. Delete notification
2. Implement:
 - a. JWT authentication
 - b. Role-based access (Admin can send to all users)
 - c. WebSocket (Socket.io) for real-time push
3. Advanced:
 - a. Store notifications in DB
 - b. Add read/unread count
 - c. Implement retry logic if WebSocket fails
 - d. Add rate limiting

◇ Frontend Requirements (React)

1. Notification Bell Component:
 - a. Unread badge count
 - b. Dropdown list
 - c. Mark as read
 - d. Infinite scroll
2. Real-time updates:
 - a. Automatically update UI on new notification
 - b. Toast alert on new notification
3. Clean architecture:
 - a. Proper folder structure
 - b. API service layer
 - c. Global state management (Redux/Zustand/Context)